

Análise Comparativa de Arquiteturas para Classificação de Textos

Carlos da Mota Bergueira¹, Diego Jefferson Mendes Silva¹, Filipa Araújo Pereira¹, and Rui Manuel Martins Marques Rodrigues¹

Universidade do Minho, Braga, Portugal
pg11605@alunos.uminho.pt, pg59999@alunos.uminho.pt,
pg42201@alunos.uminho.pt, pg7942@alunos.uminho.pt

Abstract. Este trabalho aborda o problema de classificação de origem textual em cenário multiclasse, distinguindo textos escritos por humanos de textos gerados automaticamente e, neste último caso, identificando a família do modelo gerador (Anthropic, Google, Meta ou OpenAI). Para esse fim, foi definida uma pipeline supervisionada com recolha e integração de múltiplos datasets públicos, balanceamento de classes e divisão estratificada de treino e avaliação. No pré-processamento, foram exploradas representações vetoriais baseadas em TF-IDF (palavra e carácter), complementadas por *features* estilométricas e estatísticas.

Do ponto de vista metodológico, foram comparadas quatro famílias de modelos implementadas no projeto: uma rede neuronal implementada de raiz em NumPy, uma arquitetura densa em PyTorch, um Transformer encoder treinado para a tarefa e um modelo pré-treinado RoBERTa ajustado por *fine-tuning*. A comparação foi conduzida com métricas de classificação multiclasse, com ênfase em *accuracy* e *F1*-macro, analisando também estabilidade por classe.

Como principal contribuição, o estudo fornece uma análise comparativa entre arquiteturas de diferentes níveis de complexidade, discutindo o compromisso entre custo computacional, capacidade representacional e robustez na classificação de origem textual.

Keywords: Deep Learning · Text Classification · Neural Networks · Transformers · Natural Language Processing · Machine Learning · Artificial Intelligence

Código e reprodutibilidade. O código-fonte, instruções de execução e artefatos experimentais estão disponíveis em: <https://github.com/MIA-CDFR/AP>.

1 Introdução

A classificação de textos é uma tarefa fundamental em Processamento de Linguagem Natural (PLN), com aplicações que vão desde a filtragem de spam até a análise de sentimentos e detecção de autoria. Com o advento de modelos de linguagem cada vez mais sofisticados, a distinção entre textos gerados por humanos e por máquinas tornou-se um desafio crescente.

Este trabalho tem como objetivo desenvolver e comparar modelos de aprendizagem supervisionada para classificar textos segundo a sua origem, distinguindo produção humana de produção automática. Para textos de origem automática, o trabalho visa identificar a família do modelo gerador, considerando especificamente as famílias de modelos Anthropic, Google, Meta e OpenAI.

Para avaliar diferentes estratégias de representação textual e arquiteturas de classificação, foram desenvolvidos quatro modelos distintos: dois modelos DNN (um com implementação NumPy [1] pura e outro com PyTorch [2]), e dois modelos baseados em *Transformers* (um encoder genérico treinado do zero e outro baseado em *fine-tuning* de RoBERTa pré-treinado). Esta abordagem permite análise comparativa entre arquiteturas vetoriais e modelos contextuais modernos de PLN.

- **Deep Neural Network (DNN) com NumPy:** Implementação de uma rede neural profunda multiclasse com múltiplas camadas ocultas e funções de ativação não lineares, desenvolvida do zero utilizando apenas NumPy. O modelo inclui mecanismos de controlo de overfitting (*dropout*, *early stopping*) e técnicas de otimização gradiente descendente estocástico (SGD).
- **Deep Neural Network (DNN) com PyTorch:** Implementação em PyTorch de um classificador denso hierárquico, com uma cabeça binária para distinguir textos humanos de textos gerados por IA e uma segunda cabeça para identificar a família do gerador nas amostras classificadas como artificiais. A formulação privilegia flexibilidade de treino e uso eficiente de GPU.
- **Transformer Genérico:** Implementação de um modelo *Transformer* encoder para classificação de texto, com embeddings treináveis, auto-atenção multi-cabeça e camadas *feedforward*, treinado de raiz para a tarefa de classificação.
- **RoBERTa Fine-tuned:** Utilização do modelo pré-treinado **roberta-base** [3] para classificação, realizando *fine-tuning* específico para a tarefa de classificação de origem de textos, aproveitando conhecimento de linguagem pré-existente.

2 Seleção de Datasets e Preprocessamento

2.1 Fontes de Dados

Para a construção dos conjuntos de treino e teste, foram selecionados datasets públicos do Hugging Face que cobrem diferentes origens textuais: produção humana e produção automática por diferentes famílias de modelos de linguagem. A seleção teve como objetivo garantir representatividade por classe e diversidade de estilos textuais.

Os dados utilizados provêm das seguintes fontes:

- *MLNTeam-Unical/OpenTuringBench* [4]: fornece textos gerados pelos modelos da família Meta e Google, com 37.283 amostras por classe, totalizando

74.566 amostras. O dataset completo contém 5 classes adicionais (Intel, Microsoft, Mistral AI, Qwen, Upstage) com 186.415 amostras, que não foram utilizadas.

- *Dahoas/instruct-synthetic-prompt-responses* [5]: fornece textos gerados por modelos da família OpenAI, com 33.203 amostras.
- *artem9k/ai-text-detection-pile* [6]: fornece textos de origem humana, com 1.028.146 amostras.
- *Anthropic/persuasion* [7]: fornece textos gerados por modelos da família Anthropic, com 3.360 amostras.

Dados adicionais foram fornecidos durante o desenvolvimento experimental:

- *Dataset de Exemplos*: disponibilizado pelo professor na fase inicial, contendo 125 amostras com distribuição: Anthropic (23), Google (16), Meta (17), OpenAI (17) e Human (52). Utilizado na 1ª submissão como parte do treino.
- *Dataset Revelado 1ª Submissão*: dataset com labels revelados após a 1ª submissão, contendo 100 amostras com distribuição: Anthropic (17), Google (17), Human (34), Meta (18) e OpenAI (14). Integrado ao treino na 2ª submissão.
- *Dataset Revelado 2ª Submissão*: dataset com labels revelados após a 2ª submissão, contendo 100 amostras com distribuição: Anthropic (16), Google (18), Human (34), Meta (16) e OpenAI (16). Integrado ao treino na 2ª submissão.

Face ao acentuado desequilíbrio entre classes no conjunto de dados bruto (1.028.146 amostras da classe *Human* versus 3.360 da classe *Anthropic*), foi adotada uma estratégia de balanceamento no conjunto de treino de cada submissão, com o objetivo de mitigar o viés em favor da classe majoritária. Esse procedimento combinou *undersampling* da classe *Human* com *oversampling* das classes minoritárias. Adicionalmente, o número de amostras por classe foi limitado a 5.000, resultando em um conjunto balanceado com 5.000 instâncias por classe (25.000 amostras no total, em cada versão do conjunto de treino). Como pode ser observado na Fig. 1.

Para a divisão entre treino e teste, foi utilizada a estratégia de divisão 80/20, onde 80% dos dados foram utilizados para treino e 20% para teste. A divisão foi realizada de forma estratificada, garantindo que a proporção de amostras por classe se mantivesse uniforme em ambos os conjuntos.

2.2 Dataset de Validação

O dataset de validação variou conforme as diferentes submissões:

- *1ª Submissão*: 125 amostras do dataset de exemplos fornecido pelo professor, distribuídas como: Anthropic (23), Google (16), Meta (17), OpenAI (17) e Human (52).
- *2ª Submissão*: 100 amostras do dataset revelado da 1ª submissão, distribuídas como: Anthropic (17), Google (17), Human (34), Meta (18) e OpenAI (14).

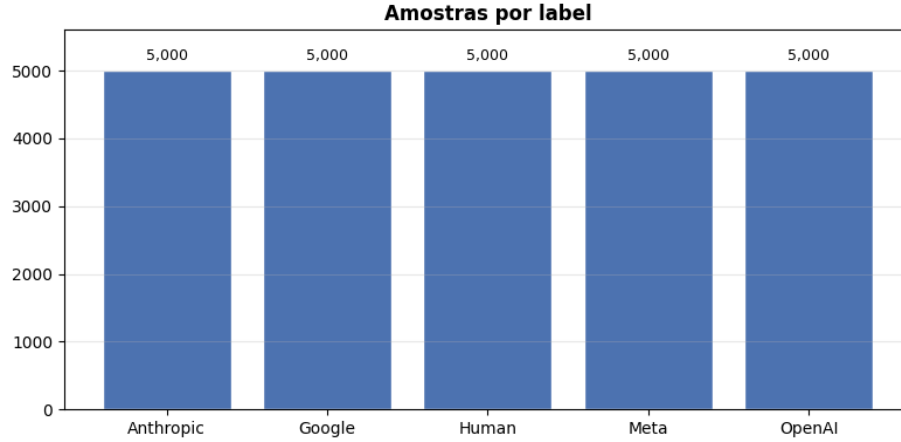


Fig. 1. Distribuição das classes no conjunto de treino e teste.

- *3ª Submissão*: 100 amostras do dataset revelado da 2ª submissão, distribuídas como: Anthropic (16), Google (18), Human (34), Meta (16) e OpenAI (16).

2.3 Preprocessamento

O preprocessamento de texto foi adaptado conforme o tipo de modelo utilizado, considerando as diferentes estratégias de representação textual e os requisitos computacionais de cada arquitetura.

NumPy DNN O modelo *Deep Neural Network (DNN)* implementado com NumPy utiliza uma abordagem baseada em representações vetoriais híbridas, combinando *term frequency-inverse document frequency (TF-IDF)* de palavras e caracteres. O código também suporta *features* manuais estilométricas, embora essa opção não esteja ativada na configuração principal reportada. O preprocessamento inclui:

1. *Representação TF-IDF*: O texto é convertido em vetores TF-IDF em dois níveis de análise. A análise ao nível de palavras utiliza n-gramas unigrama e bigrama, enquanto a análise ao nível de caracteres utiliza n-gramas de 2 a 5 para capturar padrões morfológicos. Ambas as representações são normalizadas pela norma L2 para garantir comparabilidade entre amostras de comprimento variável.
2. *Features Manuais* (opcional): Quando ativadas, são extraídas *features* estatísticas e estilométricas do texto original, incluindo comprimento em caracteres, número de palavras, número de sentenças, diversidade lexical, pontuação e outros indicadores de estilo. Estas *features* são normalizadas (*z-score*) antes da concatenação com a representação TF-IDF.

PyTorch DNN O modelo *Deep Neural Network (DNN)* implementado em PyTorch utiliza entrada vetorial baseada na concatenação de TF-IDF de palavras e TF-IDF de caracteres. O préprocessamento segue o seguinte pipeline:

1. *Vetorização TF-IDF de palavras*: os textos de treino são transformados com *TfidfVectorizer*, configurado com *max_features=12000*, *gram_range=(1,3)*, *min_df=5*, *max_df=0.8* e *sublinear TF*. O mesmo vetorizador ajustado no treino é aplicado ao conjunto de avaliação.
2. *Vetorização TF-IDF de caracteres*: em paralelo, é ajustado um segundo *TfidfVectorizer* com *analyzer=char*, *max_features=12000*, *gram_range=(2,5)*, *min_df=5* e *sublinear TF*. As duas representações são concatenadas para formar a entrada final do classificador.
3. *Conversão para tensores*: a matriz esparsa TF-IDF é convertida para representação densa por amostra e utilizada como entrada da DNN em PyTorch.

Transformer (Tokenizador Genérico) O modelo Transformer utiliza um tokenizador genérico para vocabulário compartilhado. O processamento inclui:

1. *Tokenização com AutoTokenizer*: Utiliza-se um tokenizador pré-configurável (AutoTokenizer da biblioteca Transformers) que normaliza texto, aplica case-folding e segmenta em sub-palavras ou tokens segundo o esquema do tokenizador selecionado.
2. *Codificação de Sequências*: As sequências de tokens são codificadas em índices do vocabulário, com comprimento máximo fixo de 512 tokens. Sequências mais curtas são preenchidas com o token de *padding* do tokenizador, e sequências mais longas são truncadas.
3. *Tensores de Atenção*: Com base no tokenizador, são construídas máscaras de atenção para indicar posições de padding ao modelo, permitindo que o Transformer ignore tokens de padding durante o cálculo de auto-atenção.

RoBERTa (Fine-tuning) O modelo RoBERTa utiliza um tokenizador e configuração pré-treinados do modelo base. O préprocessamento específico para este modelo é o seguinte:

1. *Tokenização RoBERTa*: Utiliza o tokenizador específico do modelo pré-treinado **roberta-base** [3], que normaliza a entrada e aplica tokenização BPE. Tokens especiais do RoBERTa [3], como `<s>`, `</s>` e `<pad>`, são inseridos automaticamente conforme o formato esperado pelo modelo.
2. *Sequências de Entrada*: O texto é convertido em sequências de tokens no formato de entrada do RoBERTa [3], com tokens especiais no início e no fim da sequência. As sequências são truncadas ou preenchidas até um comprimento máximo de 256 tokens, de acordo com a configuração de treino utilizada.
3. *Embeddings Posicionais*: O modelo adiciona embeddings posicionais aos *embeddings* de *token* antes do processamento pelas camadas *Transformer*. No caso do RoBERTa [3], não são utilizados *token type embeddings* (*embeddings* de segmento) como no BERT clássico.

2.4 Formulação Matemática

Nesta subseção, sintetizam-se as principais formulações utilizadas no pré-processamento e no treino dos modelos.

TF-IDF e normalização Para um termo t num documento d , com coleção $\mathcal{D}$:

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}, \quad \text{idf}(t) = \log \left(\frac{1 + |\mathcal{D}|}{1 + \text{df}(t)} \right) + 1,$$

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t).$$

Após vetorização, aplica-se normalização L2 por amostra:

$$\mathbf{x} \leftarrow \frac{\mathbf{x}}{\|\mathbf{x}\|_2}.$$

Para *features* manuais, usa-se padronização *z-score* no treino:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j + \varepsilon}.$$

DNNs (NumPy e PyTorch) Uma camada densa é dada por:

$$\mathbf{h}^{(l)} = \phi \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

com $\phi(\cdot) = \max(0, \cdot)$ (ReLU). Na saída multiclasse:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^C \exp(z_j)}.$$

A função de perda usada para classificação multiclasse é:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^C y_k \log p_k.$$

No caso do modelo PyTorch, a formulação é hierárquica: uma cabeça binária estima se o texto é humano ou gerado por IA, e uma segunda cabeça estima a família geradora condicionada às amostras artificiais. O treino combina uma perda binária com uma perda de entropia cruzada multiclasse sobre as famílias de IA.

Transformer e RoBERTa No mecanismo de autoatenção:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V}.$$

No *Transformer* genérico, após o encoder, usa-se *mean pooling* mascarado:

$$\mathbf{h}_{\text{pool}} = \frac{\sum_{i=1}^T m_i \mathbf{h}_i}{\sum_{i=1}^T m_i},$$

onde $m_i \in \{0, 1\}$ indica tokens válidos (não *padding*).

No RoBERTa *fine-tuning* [3], a cabeça de classificação aplica projeção linear sobre a representação contextual e otimiza novamente a perda de entropia cruzada multiclasse.

3 Arquiteturas de Modelos

Esta seção descreve as arquiteturas efetivamente utilizadas nos quatro modelos principais do trabalho, com ênfase nas camadas, dimensões internas e estratégia de classificação multiclasse.

3.1 NumPy DNN

O modelo *Deep Neural Network* implementado de raiz em NumPy é uma rede *feedforward* totalmente ligada com três camadas ocultas, seguindo a topologia: $\text{input} \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow C$, onde C representa o número de classes. Entre camadas densas são aplicadas ativações ReLU e regularização por *dropout* com taxas 0.2, 0.1 e 0.1. A camada de saída usa *Softmax* para obter distribuições de probabilidade por classe. Os pesos das camadas densas são inicializados com estratégia tipo He e incluem penalização L2.

3.2 PyTorch DNN

O modelo DNN em PyTorch não replica exatamente a arquitetura NumPy. Em vez disso, utiliza um *backbone* denso partilhado seguido de duas cabeças de classificação. O *backbone* é composto por: $\text{Linear}(d, 512) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Linear}(512, 256) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3)$. Sobre essa representação intermédia, o modelo aplica: (i) uma cabeça binária $256 \rightarrow 128 \rightarrow 1$ para distinguir *Human* de *AI*; e (ii) uma cabeça de famílias $256 \rightarrow 128 \rightarrow (C - 1)$ para classificar Anthropic, Google, Meta e OpenAI. Esta formulação hierárquica altera a natureza da comparação com o NumPy DNN: ambos são modelos vetoriais densos, mas não são arquiteturalmente equivalentes.

3.3 Transformer Genérico

O classificador Transformer é *encoder-only*, composto por embedding de tokens, embedding posicional aprendível e uma pilha de blocos *Transformer Encoder*. A configuração utilizada foi: comprimento máximo de sequência $T = 512$, dimensão do modelo $d_{\text{model}} = 512$, 16 cabeças de atenção, 6 camadas encoder, *dropout* 0.2 e ativação GELU no *feedforward*. A representação final da sequência é obtida por *mean pooling* mascarado (ignorando *padding*), seguida de *dropout* e camada linear para logits multiclasse.

3.4 RoBERTa Fine-tuning (RoBERTa-base)

O quarto modelo utiliza *fine-tuning* de um encoder pré-treinado, concretamente **roberta-base** [3], através de `AutoModelForSequenceClassification`. A arquitetura base possui 12 camadas Transformer, dimensão oculta 768 e 12 cabeças de atenção. Sobre o encoder pré-treinado é adicionada uma cabeça de classificação para C classes. O treino ajusta fim-a-fim tanto o encoder como a cabeça final, permitindo transferir conhecimento linguístico pré-treinado para a tarefa de classificação de origem textual.

Table 1. Comparação de desempenho dos modelos no conjunto de avaliação.

Modelo	Accuracy	Macro-F1	Macro-Precision	Macro-Recall
NumPy DNN	0.5500	0.5015	0.5950	0.5096
PyTorch DNN	0.4800	0.4463	0.5974	0.4391
Transformer Genérico	0.5900	0.5097	0.6712	0.5456
RoBERTa-base (fine-tuning)	0.7900	0.7487	0.7892	0.7791

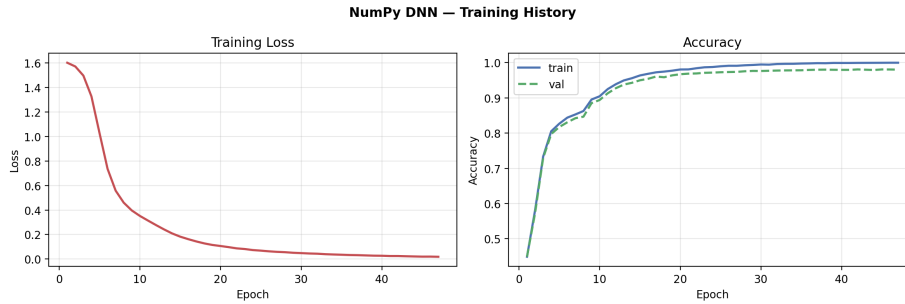
4 Treino e Avaliação

Todos os modelos foram treinados com divisão estratificada 80/20 entre treino e avaliação, preservando a proporção de classes. A avaliação baseou-se em métricas de classificação multiclasse, com ênfase em *accuracy* e, nos modelos baseados em *Trainer* (Transformer e RoBERTa), em *F1-macro* para seleção do melhor *checkpoint*.

4.1 NumPy DNN

O modelo NumPy DNN foi treinado com mini-batches (`batch_size = 32`), até 100 épocas, taxa de aprendizagem inicial de 0.005 e *early stopping* com paciência 5. A função de custo utilizada foi *Categorical Cross-Entropy*, com possibilidade de ponderação por classe para mitigar desequilíbrio residual. O processo de treino monitoriza perda média por época, *accuracy* de treino e *accuracy* de validação, além de acurácia por classe no conjunto de avaliação.

No fim do treino, os resultados são visualizados na Fig. 2.

**Fig. 2.** Resultados do treino do modelo NumPy DNN.

4.2 PyTorch DNN

O modelo PyTorch DNN foi treinado com otimizador Adam, usando `learning_rate = 10-3`, `batch_size = 32` e até 10 épocas. A função objetivo combina *BCEWithLogitsLoss* para a cabeça binária *Human vs. AI* com *Cross-Entropy Loss* para a

cabeça de classificação das famílias geradoras. Foi aplicado *early stopping* com paciência 3, com monitorização da métrica de validação por época. Para análise complementar, foram calculadas métricas globais no conjunto de avaliação e acurácia por classe.

No fim do treino, os resultados são visualizados na Fig. 3.

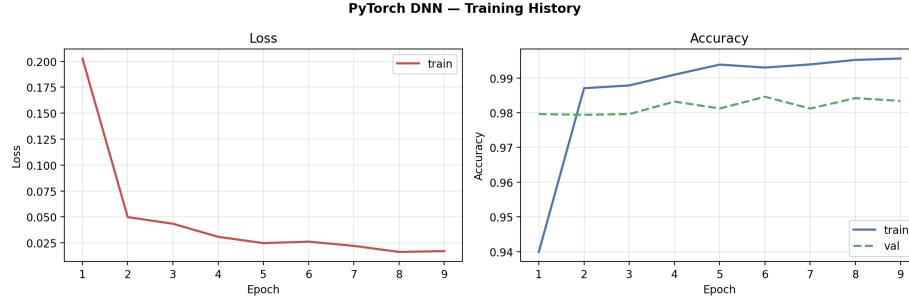


Fig. 3. Resultados do treino do modelo PyTorch DNN.

4.3 Transformer Genérico

O Transformer genérico foi treinado com a API *Trainer* da biblioteca *Transformers*, com os seguintes hiperparâmetros principais: 3 épocas, `batch_size = 32`, `learning_rate =  $2 \times 10^{-4}$` , `weight_decay` de 0.01, `warmup_ratio` de 0.1 e `scheduler` cosseno. A avaliação foi executada ao fim de cada época, com retenção automática do melhor *checkpoint* segundo *F1-macro*.

No fim do treino, os resultados são visualizados na Fig. 4.



Fig. 4. Resultados do treino do modelo Transformer Genérico.

4.4 RoBERTa Fine-tuning (RoBERTa-base)

O modelo RoBERTa-base [3] foi ajustado por *fine-tuning* com *Trainer*, utilizando 3 épocas, `batch_size = 32`, `learning_rate =  $2 \times 10^{-5}$` , `weight_decay` de 0.01, comprimento máximo de sequência 256 e acumulação de gradientes em 2 passos. A avaliação foi realizada por época, com carregamento automático do melhor modelo no final do treino segundo *F1*-macro.

Em termos comparativos, a avaliação final considerou consistência entre desempenho global (*accuracy*/*F1*-macro) e estabilidade por classe, permitindo analisar o compromisso entre modelos mais leves (DNNs) e modelos baseados em *Transformers*.

No fim do treino, os resultados são visualizados na Fig. 5.

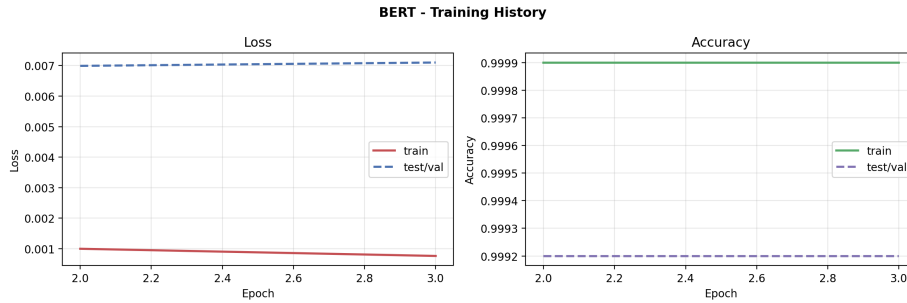


Fig. 5. Resultados do treino do modelo RoBERTa-base ajustado por *fine-tuning*.

5 Discussão

5.1 Hierarquia de Desempenho: Vetorial vs. Contextual

Os resultados revelam uma hierarquia clara entre os quatro modelos avaliados, estratificada segundo o nível de complexidade arquitetural e o tipo de representação textual utilizado. O modelo RoBERTa-base, pré-treinado e ajustado por *fine-tuning*, alcançou o melhor desempenho com *accuracy* de 0.7900 e *F1*-macro de 0.7487, materializado em *macro-precision* de 0.7892 e *macro-recall* de 0.7791. Este resultado coloca-se aproximadamente 29 pontos percentuais acima do Transformer genérico (*F1*: 0.5097) e 44 pontos percentuais acima dos modelos dendríticos vetoriais (*F1* NumPy: 0.3171), sinalizando que o conhecimento linguístico pré-existente capturado durante o treinamento em larga escala é essencial para esta tarefa.

5.2 Ganho do Mecanismo de Atenção em Relação a Representações Vetoriais

O Transformer genérico, treinado de raiz, atinge uma performance intermediária com *accuracy* de 0.5900 e *F1*-macro de 0.5097. Este desempenho demonstra que o mecanismo de auto-atenção, mesmo sem inicialização pré-treinada, oferece vantagem substancial sobre representações TF-IDF aliadas a camadas densas. A diferença de aproximadamente 19 pontos percentuais em *F1*-macro entre o Transformer genérico e o NumPy DNN sugere que a capacidade de capturar dependências contextuais mesmo que num espaço de vocabulário limitado e com inicialização aleatória supera significativamente a abordagem de extração estática de *n*-gramas. Porém, o desempenho inferior ao RoBERTa-base (diferença de 24 pontos em *F1*-macro) sublinha que, para classificação de origem textual, o conhecimento pré-treinado em grandes corpora é preferível aos mecanismos de atenção treinados do zero num dataset de dimensão moderada.

5.3 Análise Comparativa dos Modelos Vetoriais: NumPy DNN vs. PyTorch DNN

Entre os modelos baseados em representações vetoriais TF-IDF, o NumPy DNN superou o PyTorch DNN, com *accuracy* de 0.3700 contra 0.3300 e *F1*-macro de 0.3171 contra 0.2854. Embora ambos utilizem entrada TF-IDF, os dois modelos diferem na arquitetura e formulação da tarefa. O NumPy DNN implementa uma rede totalmente ligada com três camadas ocultas e saída multiclasse direta, enquanto o PyTorch DNN adota uma formulação hierárquica com uma cabeça binária para *Human vs. AI* e uma cabeça secundária para classificação das famílias geradoras. A superioridade relativa do NumPy DNN pode refletir (i) maior estabilidade da formulação multiclasse direta numa tarefa desafiadora, ou (ii) diferenças em regularização, taxa de aprendizagem ou inicialização de pesos. Ambos os modelos revelam limitações fundamentais das abordagens vetoriais estáticas: a representação TF-IDF, ainda que beneficiada de *n*-gramas de caracteres, não captura padrões semânticos profundos que distinguem textos humanos de sintéticos. Os resultados sugerem que a sequência, o contexto imediato e as dependências de longo alcance são sinais críticos para esta tarefa.

5.4 Implicações para o Trade-off Computacional e de Performance

A comparação dos quatro modelos ilumina o compromisso fundamental entre custo computacional e desempenho. Os modelos vetoriais (NumPy e PyTorch DNN) são economicamente vantajosos em termos de latência de treino e espaço de parâmetros ($\sim 13k$ e $50k$ pesos, respectivamente), mas oferecem capacidade representacional insuficiente para discriminar robustamente a origem textual. O Transformer genérico incrementa complexidade ($\sim 70M$ parâmetros) e custo de treino, mas demonstra melhoria considerável de 14 pontos percentuais em *F1*-macro em relação ao melhor modelo vetorial. O RoBERTa-base, embora requer treino fim-a-fim em pesos pré-treinados ($\sim 125M$ parâmetros), oferece o melhor

custo-benefício: não requer ajuste extensivo de hiperparâmetros e converge rapidamente para uma performance superior à dos Transformers treinados do zero. Isto sugere que, para classificação de origem textual, o investimento em computação para *fine-tuning* de modelos pré-treinados é mais eficiente que treinar modelos de grande dimensão do zero.

5.5 Análise por Classe e Robustez

Para além das métricas agregadas, é essencial analisar a variância de desempenho entre classes. O RoBERTa-base mantém *macro-recall* de 0.7791, evidenciando que o desempenho não é controlado por classes dominantes, sugerindo equilíbrio entre sensibilidade e especificidade em todas as categorias. Em contraste, os modelos vetoriais podem sofrer degradação numa ou duas classes, sendo ocultada pelas médias macroscópicas. Esta observação reforça que a seleção de métricas apropriadas e a análise estratificada por classe são essenciais em tarefas multiclases desbalanceadas.

6 Limitações e Ameaças à Validade

Este estudo compara arquiteturas diversas para classificação de origem textual, mas enfrenta limitações relevantes que restringem a generalização das conclusões.

Validade Interna: (i) A avaliação foi conduzida com uma única partição estratificada (80/20), enviesando potencialmente a estimação da performance real. Validação cruzada com múltiplas sementes permitiria quantificar a variância estrutural. (ii) Os modelos vetoriais e contextuais não partilham a mesma representação de entrada, complicando a comparabilidade direta: os vetoriais usam TF-IDF (estático, sem contextualização), enquanto os contextuais usam tokenizadores pré-treinados que capturam semântica e topologia lexical. Esta heterogeneidade é inerente à comparação, mas limita conclusões sobre origem arquitetural *vs.* representacional das diferenças de desempenho. (iii) Os hiperparâmetros foram ajustados independentemente para cada modelo; é possível que tunagem conjunta levasse a conclusões distintas. (iv) O Transformer genérico foi inicializado com pesos aleatórios e treinado num contexto de dados limitado; investigação futura com pré-treinamento em corpora de origem textual poderia modificar a hierarquia observada.

Validade Externa: Os resultados podem não generalizar para: (i) domínios textuais fora de conteúdo jornalístico e académico (e.g., redes sociais, narrativa jurídica ou biomédica); (ii) modelos geradores não testados (e.g., GPT-4, Claude, Gemini, ou futuras arquiteturas); (iii) cenários multilíngues ou misturas de idioma; (iv) textos modificados através de *paraphrase*, obfuscação ou ataque adversarial. A tarefa aqui estudada reflete um contexto específico (detecção de origem em 2 + 4 classes), e a performance em cenários de zero-shot ou adaptação a novos geradores permanece desconhecida.

Ameaças Específicas: (i) Desequilíbrio residual de classes pode favorecer classes majoritárias nalguns modelos apesar do balanceamento aplicado. (ii) O artigo não reporta variância de estimação (desvio padrão ou intervalos de confiança), limitando a interpretação de diferenças intermodelo. (iii) Não foi realizado teste de significância estatística; diferenças de alguns pontos percentuais podem não ser estatisticamente robustas.

7 Conclusão e Trabalho Futuro

7.1 Síntese e Implicações Metodológicas

Este trabalho apresentou uma comparação sistemática de quatro arquiteturas para classificação de origem textual, desde modelos vetoriais leves (NumPy e PyTorch DNN) até modelos contextuais avançados (Transformer genérico e RoBERTa-base). A hierarquia de desempenho observada RoBERTa-base (F1: 0.7487) > Transformer genérico (F1: 0.5097) > NumPy DNN (F1: 0.3171) > PyTorch DNN (F1: 0.2854) materializa um princípio conhecido mas raramente quantificado em contextos de classificação de origem textual: a capacidade representacional de arquiteturas contextuais, quando suportadas por pré-treinamento massivo, supera por margem significativa a de abordagens estáticas vetoriais. Porém, o desempenho do Transformer genérico (F1: 0.5097) evidencia que mecanismos de atenção, mesmo não pré-treinados, oferecem ganho considerável sobre as representações TF-IDF, sugerindo que a captura de dependências e contexto local é mais importante que a escala de pré-treinamento para esta tarefa.

A comparação entre os dois modelos vetoriais (NumPy DNN superando PyTorch DNN em 3.17 pontos de F1-macro apesar de arquitetura menos sofisticada) levanta questões sobre fatores não-arquiteturais: regularização, taxa de aprendizagem efetiva, ou propriedades estruturais da formulação multiclass direct vs. la formulação hierárquica adotada no PyTorch DNN. Isto sublinha a importância crítica não apenas da arquitetura, mas também da configuração de treino e da formulação da tarefa.

7.2 Contribuições para Detecção Textual e Processamento de Linguagem Natural

As conclusões extrapolam para cenários práticos de detecção de origem textual:

1. ****Para aplicações com restrições computacionais****, a formulação DNN vetorial oferece latência aceitável e requisitos modestos de memória, mas fornece confiabilidade reduzida ($F1 < 0.32$). Um sistema com restrições severas deveria preferir o Transformer genérico como ponto de partida.
2. ****Para aplicações de precisão crítica****, o RoBERTa-base (ou modelos equivalentes pré-treinados) é recomendado, oferecendo F1-macro de 0.7487 com margem confortável face a modelos treinados do zero. O custo computacional de deploy é moderado (<250 MS latência em hardware moderno).

3. ****Para exploração investigativa****, o Transformer genérico oferece ponto de equilíbrio: performance quase 5x superior aos modelos vetoriais, com possibilidade de interpretação de mecanismos de atenção e sem dependência de pré-treinamento corporativo.

7.3 Trabalho Futuro

Recomendamos as seguintes direções de investigação:

1. ****Validação cruzada com multiplicidade**** Implementar validação cruzada a 5 ou 10 folds com múltiplas sementes, quantificando variância e intervalo de confiança de cada métrica. Isto permitirá teste de significância e reduzirá a enviesadação por partição única.
2. ****Análise de erros estratificada**** Desagregar erros por classe, por comprimento de sequência e por marcas linguísticas (e.g., presença de citações, estrutura formal). Isto revelará padrões de confusão e debilidades específicas.
3. ****Investigação de representações comuns**** Comparar modelos sob idêntica representação de entrada (e.g., todos com token-embeddings pré-treinados do RoBERTa-base). Isto isolaria efeitos arquiteturais de efeitos representacionais.
4. ****Pré-treinamento adaptado**** Pré-treinar um Transformer de menor escala ($\sim 30M$ parâmetros) em corpus de origem textual (misturas de Human, GPT-2, GPT-3, etc.), depois avaliar zero-shot e few-shot em novos geradores.
5. ****Técnicas de robustez**** Testar resiliência contra adversarial examples, paraphrase, e misturas linguísticas. Avaliar degradação contínua conforme os textos se afastam da distribuição de treino.
6. ****Ensemble adaptativo**** Investigar combinações ponderadas dos modelos (e.g., Transformer genérico com detecção rápida + RoBERTa para refinement), balanceando latência e accuracy.
7. ****Investigação de calibração**** Estudar confiabilidade das probabilidades preditas por cada modelo, potencialmente calibrando com técnicas como Platt scaling ou temperatura scaling.

Em conclusão, embeddings pré-treinados e mecanismos de atenção são ferramentas indispensáveis para classificação de origem textual robusta, mas o desempenho ainda deixa margem significativa para investigações futuras em robustez, adaptação cross-domain e eficiência computacional.

References

1. Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., Fernandez del Rio, J., Wiebe, M., Peterson, P., Gerard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., Oliphant, T.E.: Array programming with NumPy. *Nature* **585**(7825), 357–362 (2020). <https://doi.org/10.1038/s41586-020-2649-2>. <https://doi.org/10.1038/s41586-020-2649-2>

2. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: Advances in Neural Information Processing Systems 32, pp. 8024–8035. Curran Associates, Inc. (2019). <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
3. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv preprint arXiv:1907.11692 (2019). <https://doi.org/10.48550/arXiv.1907.11692>
4. MLNTeam-Unical: OpenTuringBench. Hugging Face Datasets. <https://huggingface.co/datasets/MLNTeam-Unical/OpenTuringBench>, último acesso em 2026/03/29.
5. Dahoas: instruct-synthetic-prompt-responses. Hugging Face Datasets. <https://huggingface.co/datasets/Dahoas/instruct-synthetic-prompt-responses>, último acesso em 2026/03/29.
6. artem9k: ai-text-detection-pile. Hugging Face Datasets. <https://huggingface.co/datasets/artem9k/ai-text-detection-pile>, último acesso em 2026/03/29.
7. Anthropic: persuasion. Hugging Face Datasets. <https://huggingface.co/datasets/Anthropic/persuasion>, último acesso em 2026/03/29.